

LONG-PREP

Long-Prep é um jogo desenvolvido com o objetivo de aplicar e demonstrar conceitos de Programação Orientada a Objetos. Sua história acompanha um herói que teve seu progresso apagado após ser derrotado pelo chefe final, sendo obrigado a reiniciar sua jornada e tentar novamente alcançar a fase 100.

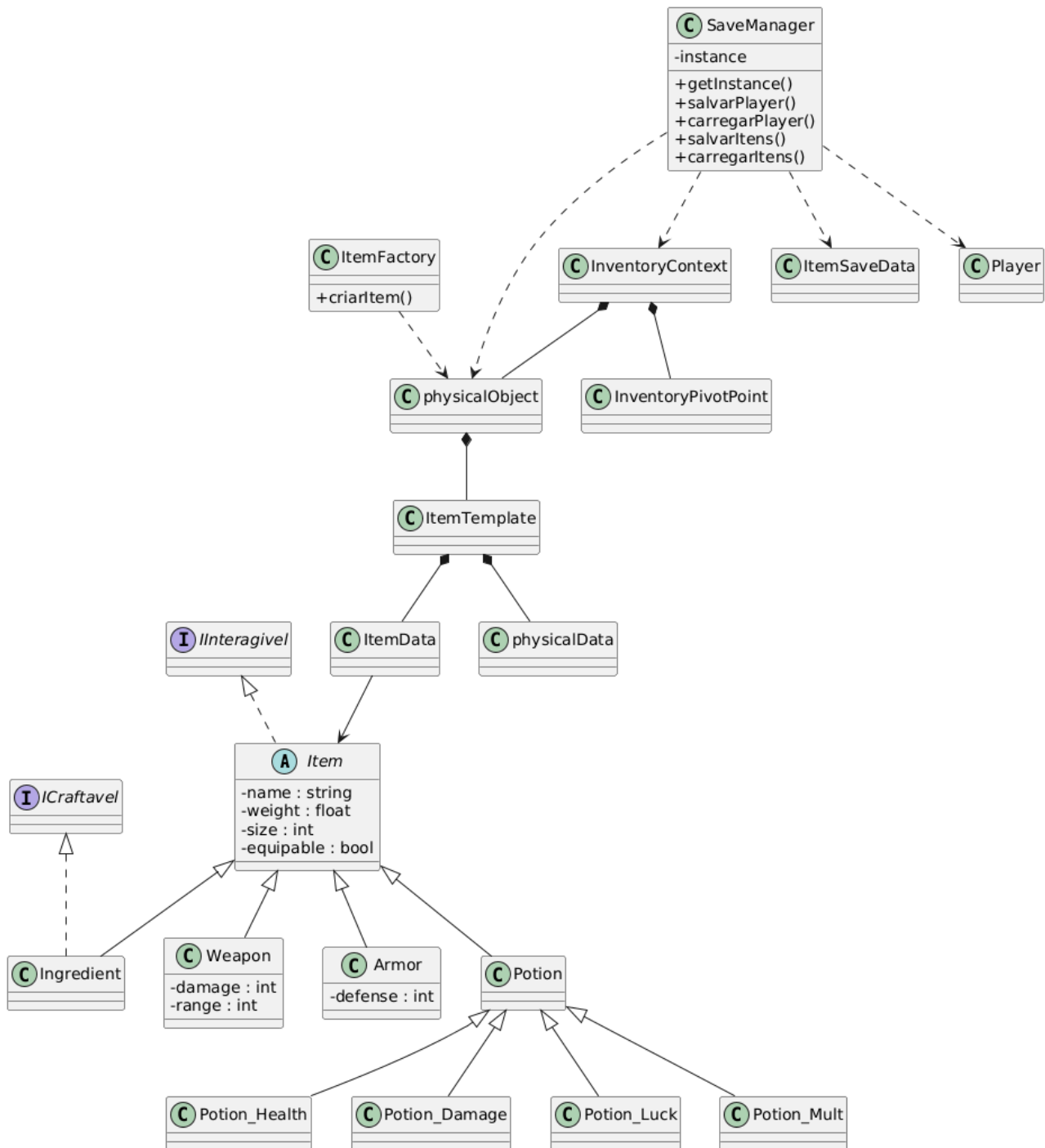
O projeto utiliza diversas técnicas e tecnologias relacionadas ao desenvolvimento de software, incluindo conceitos de POO, persistência de dados e integração com bibliotecas externas.

Raylib e Box2D

O Long-Prep utiliza a biblioteca Raylib para renderização gráfica e criação da interface visual do jogo. Para a simulação física dos objetos foi utilizada a biblioteca Box2D.

A separação entre a representação gráfica e a simulação física permitiu uma arquitetura mais organizada, reduzindo bugs e facilitando a manutenção do código.

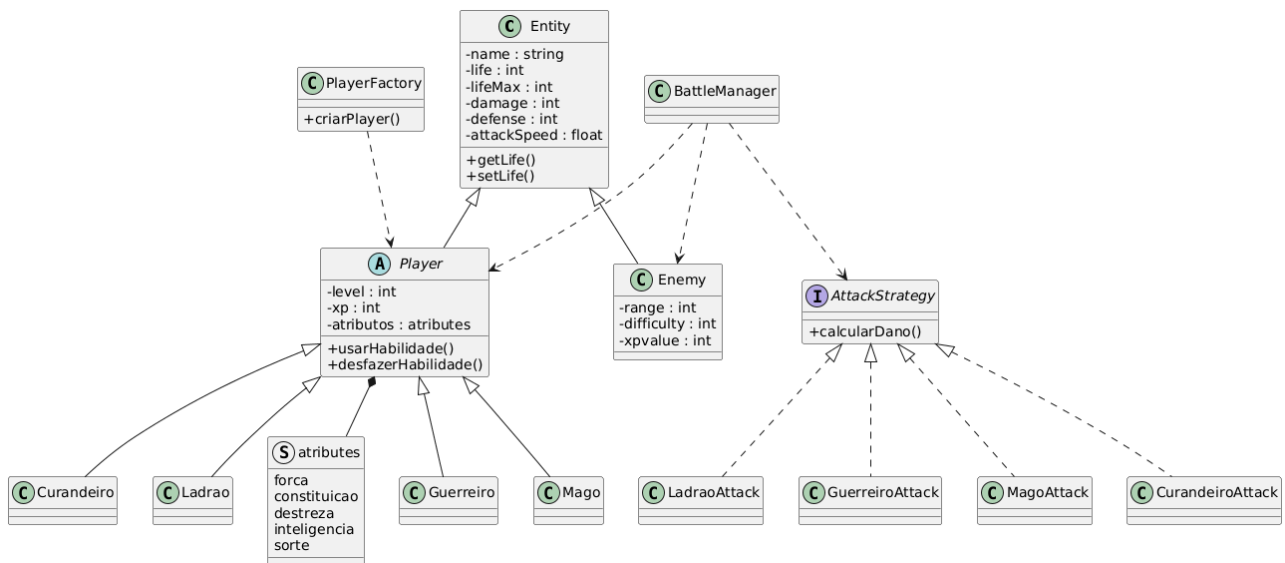
Arquitetura Geral Dos Itens e Save



O sistema de itens foi desenvolvido utilizando composição e herança, permitindo a criação de diferentes categorias de itens sem duplicação de código.

O sistema de save foi integrado ao SQLite, possibilitando o armazenamento persistente do progresso do jogador.

Arquitetura Geral Das Entidades



A arquitetura das entidades foi construída utilizando herança, permitindo que atributos e comportamentos comuns fossem compartilhados entre diferentes tipos de objetos do jogo.

A classe Entity atua como classe base do sistema, sendo herdada pelas classes Player e Enemy. A classe Player é abstrata e serve como base para as diferentes classes jogáveis do jogo.

Conceitos De POO Utilizados

Encapsulamento:

O encapsulamento foi utilizado em praticamente todas as classes do projeto. Diversos atributos, como vida de inimigos, atributos do jogador e propriedades dos itens, foram declarados como privados e acessados por meio de métodos públicos.

Essa abordagem aumenta a segurança do código e reduz o risco de modificações indevidas.

Herança:

A herança foi amplamente utilizada para evitar repetição de código e facilitar a manutenção do projeto.

Um exemplo é a classe Entity, que centraliza atributos compartilhados por jogadores e inimigos.

Polimorfismo:

O polimorfismo foi aplicado principalmente no sistema de habilidades do jogador.

Todas as classes jogáveis implementam o método:

```
usarHabilidade()
```

porém cada classe executa uma ação diferente, permitindo comportamentos distintos através da mesma interface.

Abstracao:

A classe Player foi implementada como uma classe abstrata, impossibilitando sua instanciação direta.

Dessa forma, o jogador deve obrigatoriamente selecionar uma das classes concretas disponíveis no jogo.

Padroes De Projeto Utilizados

Singleton:

O padrão Singleton foi utilizado na classe SaveManager.

Seu objetivo é garantir que exista apenas uma instância responsável pelo gerenciamento dos dados salvos durante toda a execução do programa.

Factory:

O padrão Factory foi utilizado na criação de jogadores e itens.

A PlayerFactory é responsável por criar as diferentes classes jogáveis disponíveis no jogo.

A ItemFactory centraliza a criação dos objetos físicos e lógicos dos itens, reduzindo a duplicação de código e simplificando a manutenção.

Strategy:

O padrão Strategy foi utilizado no sistema de combate.

Cada classe possui uma estratégia própria para cálculo de dano, permitindo que novas formas de combate sejam adicionadas sem modificar o restante do sistema.

Sistema De Persistencia

O sistema de persistência utiliza SQLite para armazenar informações do jogo.

Entre os dados armazenados estão:

- Informações do jogador;
- Inventário;
- Equipamentos;
- Fase atual;
- Progresso geral da partida.

Ao carregar um save, os dados armazenados substituem o estado atual do jogo, restaurando a sessão salva anteriormente.

Gtests

O projeto utiliza Google Test (GTest) para validação de componentes importantes do sistema.

Os testes foram aplicados principalmente às classes relacionadas ao jogador e aos itens, verificando comportamentos esperados, funcionamento de métodos e tratamento de exceções.

Principais Desafios

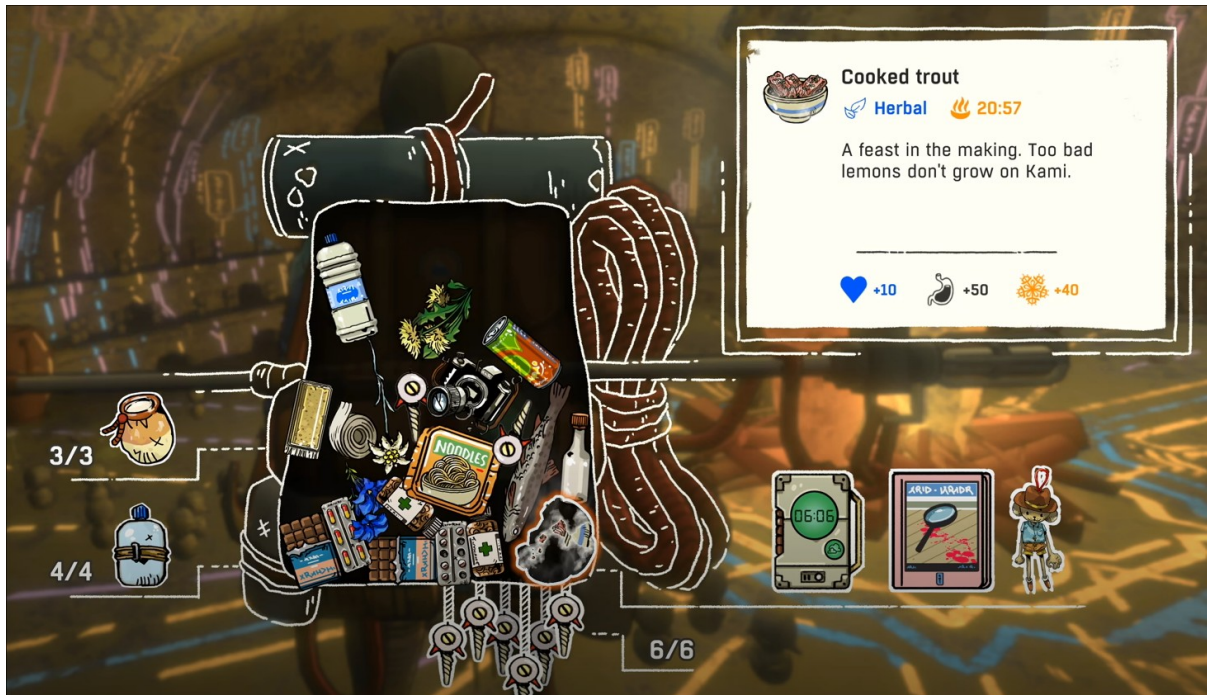
O desenvolvimento do Long-Prep apresentou diversos desafios.

Além da necessidade de aprender e aplicar conceitos de Programação Orientada a Objetos, foi necessário estudar o funcionamento das bibliotecas Raylib e Box2D para integrar corretamente os sistemas gráfico e físico.

Outro desafio significativo foi a implementação do polimorfismo. Quando esse requisito foi adicionado ao projeto, grande parte do sistema já estava concluída, exigindo uma reestruturação considerável da arquitetura para acomodar as novas classes e padrões de projeto.

Inspiracoes

A ideia inicial do Long-Prep surgiu a partir do jogo Cairn, especialmente devido ao seu sistema de inventário baseado em espaço físico.



Durante as etapas iniciais do desenvolvimento, o projeto também possuía inspiração visual no jogo Sol Cesto. Inicialmente o combate seria automático e o jogador apenas escolheria os caminhos dentro da masmorra.



Após testes realizados pela equipe, decidiu-se remover essa mecânica e implementar um sistema de combate manual, proporcionando maior interação ao jogador.

Conclusao

O desenvolvimento do Long-Prep foi uma experiência de grande aprendizado, permitindo a aplicação prática de conceitos fundamentais de Programação Orientada a Objetos.

Durante o projeto foram utilizados conceitos como encapsulamento, herança, polimorfismo e abstração, além de padrões de projeto como Singleton, Factory e Strategy.